

Лабораторная работа № 5.

Интеграция frontend и backend

Тема: Fetch API, единый API-клиент, JWT и protected requests

Цель: Связать frontend Internet Shop с Express REST API: реализовать login через fetch(), общий apiFetch(), передачу Bearer JWT, загрузку защищённых данных и обработку 401/403.

Дисциплина: Node.js и Express.js

Исходные условия

Продолжайте проект express-shop. Backend должен поддерживать POST /api/auth/login, GET /api/auth/me, GET /api/orders/my, JWT access token и middleware requireAuth.

Frontend можно разместить в отдельной папке проекта и открыть через локальный web-сервер. В этой работе access token хранится только в памяти JavaScript.

Не используйте localStorage для JWT

В учебной реализации не сохраняйте access token в localStorage или sessionStorage. Token будет жить только до перезагрузки страницы.

1. Создайте структуру frontend

Структура каталогов

```
frontend/  
  index.html  
  app.js  
  api.js  
  authState.js  
  config.js  
  styles.css
```

2. Создайте config.js

```
frontend/config.js  
  
export const API_URL = "http://localhost:3000/api";
```

Если backend работает на другом host или port, измените только значение API_URL.

3. Создайте authState.js

```
frontend/authState.js  
  
let accessToken = null;  
let currentUser = null;  
  
export function setAuth(data) {  
  accessToken = data.accessToken;  
  currentUser = data.user;  
}
```

```

}

export function clearAuth() {
  accessToken = null;
  currentUser = null;
}

export function getAccessToken() {
  return accessToken;
}

export function getCurrentUser() {
  return currentUser;
}

```

In-memory state

После перезагрузки страницы `accessToken` и `currentUser` будут потеряны. Для этой лабораторной это ожидаемое поведение.

4. Создайте `APIError`

```

frontend/api.js

export class APIError extends Error {
  constructor(message, status) {
    super(message);
    this.status = status;
  }
}

```

5. Реализуйте `apiFetch()`

```

frontend/api.js

import { API_URL } from "../config.js";
import {
  getAccessToken,
  clearAuth
} from "../authState.js";

export async function apiFetch(
  path,
  { body, headers = {}, ...options } = {}
) {
  const token = getAccessToken();

  const response = await fetch(`${API_URL}${path}`, {
    ...options,
    headers: {
      ...(body ? { "Content-Type": "application/json" } : {}),
      ...(token ? { Authorization: `Bearer ${token}` } : {}),
      ...headers
    },
    body: body ? JSON.stringify(body) : undefined
  });
}

```

```

let data = null;

if (response.status !== 204) {
  data = await response.json();
}

if (response.status === 401) {
  clearAuth();
}

if (!response.ok) {
  throw new APIError(
    data?.error ?? `HTTP ${response.status}`,
    response.status
  );
}

return data;
}

```

6. Разберите работу apiFetch()

1. path добавляется к API_URL.
2. Если есть body, автоматически устанавливается Content-Type: application/json.
3. Если есть access token, автоматически добавляется Authorization: Bearer <token>.
4. Для body вызывается JSON.stringify().
5. HTTP status 401 очищает локальное auth state.
6. При response.ok === false создаётся APIError.

7. Создайте HTML интерфейс

```

frontend/index.html

<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Internet Shop</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <main>
    <section id="guest-section">
      <h1>Вход</h1>

      <form id="login-form">
        <input
          id="email"
          type="email"
          placeholder="Email"
          required
        >

        <input
          id="password"
          type="password"
          placeholder="Password"
          required

```

```

    >

    <button id="login-button" type="submit">
      Войти
    </button>
  </form>

  <p id="login-error"></p>
</section>

<section id="user-section" hidden>
  <h2 id="user-name"></h2>
  <button id="orders-button">Мои заказы</button>
  <button id="logout-button">Выйти</button>
  <p id="orders-status"></p>
  <div id="orders-list"></div>
</section>
</main>

<script type="module" src="app.js"></script>
</body>
</html>

```

8. Реализуйте login()

```

frontend/app.js

import { apiFetch, APIError } from "../api.js";
import {
  setAuth,
  clearAuth
} from "../authState.js";

async function login(email, password) {
  const data = await apiFetch("/auth/login", {
    method: "POST",
    body: {
      email,
      password
    }
  });
};

setAuth(data);
return data.user;
}

```

9. Реализуйте переключение интерфейса

```

frontend/app.js

const guestSection = document.querySelector("#guest-section");
const userSection = document.querySelector("#user-section");
const userName = document.querySelector("#user-name");

function showGuestUI() {
  guestSection.hidden = false;
  userSection.hidden = true;
  userName.textContent = "";
}

```

```

}

function showAuthenticatedUI(user) {
  guestSection.hidden = true;
  userSection.hidden = false;
  userName.textContent = `Здравствуйте, ${user.name}`;
}

```

10. Подключите submit формы

```

frontend/app.js

const form = document.querySelector("#login-form");
const emailInput = document.querySelector("#email");
const passwordInput = document.querySelector("#password");
const loginButton = document.querySelector("#login-button");
const loginError = document.querySelector("#login-error");

form.addEventListener("submit", async (event) => {
  event.preventDefault();

  loginError.textContent = "";
  loginButton.disabled = true;

  try {
    const user = await login(
      emailInput.value,
      passwordInput.value
    );

    showAuthenticatedUI(user);
    form.reset();
  } catch (error) {
    loginError.textContent = error.message;
  } finally {
    loginButton.disabled = false;
  }
});

```

11. Проверьте успешный login

7. Запустите backend.
8. Откройте frontend через локальный web-сервер.
9. Введите существующий email и правильный password.
10. Откройте DevTools → Network.
11. Найдите `POST /api/auth/login`.
12. Проверьте status 200.
13. Убедитесь, что UI показывает имя пользователя.

12. Проверьте неверные credentials

Введите неправильный password. Backend должен вернуть 401, а frontend — показать сообщение об ошибке и оставить guest UI.

13. Реализуйте загрузку своих заказов

```
frontend/app.js

async function loadMyOrders() {
  return apiFetch("/orders/my");
}
```

14. Добавьте отображение orders

```
frontend/app.js

const ordersButton = document.querySelector("#orders-button");
const ordersStatus = document.querySelector("#orders-status");
const ordersList = document.querySelector("#orders-list");

ordersButton.addEventListener("click", async () => {
  ordersStatus.textContent = "Загрузка...";
  ordersList.innerHTML = "";

  try {
    const orders = await loadMyOrders();

    ordersStatus.textContent = `Заказов: ${orders.length}`;

    for (const order of orders) {
      const element = document.createElement("div");
      element.textContent = `Заказ #${order.id}: ${order.status}`;
      ordersList.append(element);
    }
  } catch (error) {
    ordersStatus.textContent = error.message;

    if (error instanceof APIError && error.status === 401) {
      showGuestUI();
    }
  }
});
```

15. Проверьте Authorization header

14. Выполните успешный login.
15. Нажмите кнопку Мои заказы.
16. Откройте запрос GET /api/orders/my в DevTools.
17. Проверьте наличие заголовка Authorization.
18. Убедитесь, что значение начинается с Bearer .
19. Проверьте status 200 и JSON со своими заказами.

16. Проверьте 401

Для проверки установите на backend очень короткий срок действия JWT, выполните login, дождитесь expiration и повторно загрузите заказы.

- Backend должен вернуть 401.
- apiFetch() должен вызвать clearAuth().
- Frontend должен показать guest UI.
- После теста верните нормальный срок действия access token.

17. Реализуйте logout

frontend/app.js

```
const logoutButton = document.querySelector("#logout-button");

logoutButton.addEventListener("click", () => {
  clearAuth();
  ordersList.innerHTML = "";
  ordersStatus.textContent = "";
  showGuestUI();
});
```

18. Проверьте reload страницы

После успешного login нажмите Refresh в браузере. Пользователь должен снова увидеть guest UI, потому что access token хранится только в памяти JavaScript.

Это не ошибка

Сохранение Authentication между перезагрузками требует отдельной архитектуры. На следующей лекции будет разобрано состояние аутентификации на frontend.

19. Обработка 403

Если frontend вызывает admin endpoint с JWT обычного пользователя, backend должен вернуть 403. В этом случае не вызывайте `clearAuth()`: пользователь по-прежнему аутентифицирован.

Пример обработки

```
try {
  await apiFetch("/products", {
    method: "POST",
    body: productData
  });
} catch (error) {
  if (error.status === 403) {
    alert("Недостаточно прав");
  }
}
```

20. Самостоятельное задание

20. Добавьте кнопку загрузки текущего пользователя через GET `/api/auth/me`.
21. Добавьте загрузку публичного каталога через GET `/api/products`.
22. Показывайте отдельные состояния loading, success и error.
23. Проверьте корректную обработку 401.
24. Проверьте корректную обработку 403.
25. Не дублируйте Bearer header в компонентах — используйте только `apiFetch()`.
26. Не сохраняйте JWT в `localStorage` или `sessionStorage`.
27. Добавьте logout и очистку интерфейса.

Требования к результату

- Создан единый `API_URL`.
- Создан `apiFetch()`.
- Login выполняется через Fetch API.
- Frontend проверяет HTTP status.
- User и accessToken сохраняются в runtime state.
- Protected requests автоматически получают Bearer token.
- `GET /api/orders/my` работает после login.
- 401 очищает auth state.
- 403 не завершает Authentication.
- Реализован logout.
- JWT не хранится в localStorage.

Критерии оценивания

№	Критерий
1	Создан config.js
2	Создан authState.js
3	Создан единый apiFetch()
4	apiFetch устанавливает Content-Type для JSON
5	apiFetch добавляет Bearer token
6	apiFetch проверяет response.ok
7	Login form отправляется через fetch()
8	Успешный login изменяет UI
9	Ошибка login отображается пользователю
10	Protected orders загружаются после login
11	401 очищает auth state
12	403 обрабатывается без logout
13	Logout очищает User и token
14	JWT не хранится в localStorage/sessionStorage
15	Студент объясняет границу ответственности frontend и backend